

We cannot grow what we cannot measure.

Three systems are watching and none of them sees a whole customer. GA4 and the Meta Pixel run on `nursia.io` ; PostHog runs on `app.nursia.io` ; the redirect between them discards the query string, so no identifier survives the hop. The event every campaign is supposed to bid toward, `CompleteRegistration` , exists in neither. Until that is fixed, every budget decision is a guess wearing a number. This PRD specifies the estate area by area, sets numeric gates for done, names the technical dependencies, and adds the performance budget those pages have to stay inside while it happens.

STATUS Draft for sprint planning	MARKETING SITE nursia.io · this repo	PRODUCT APP app.nursia.io · external	POSTHOG PROJECT NCLEX-Nursia · 595036
SPRINT SIZE 79 points	P0 ITEMS 13 of 36		

EXECUTIVE SUMMARY

Nursia is buying traffic it cannot measure

0 Registrations attributable to a campaign. The primary Google Ads conversion has recorded nothing since the app split.	0 Systems that can see a whole customer. Three are installed; each sees one half.	49 Stored analytics events holding a live auth token. A credential exposure, not a measurement gap.
---	---	---

4461ms

p75 LCP on the page every paid click lands on, against a 2,500 ms threshold.

Signup, login and practice moved to `app.nursia.io` . The handoff is a bare server redirect that discards the query string, so every ad identifier — `gclid` , `fbclid` , all five UTMs — dies at the boundary. The code that reported a signup now sits behind that redirect and never runs. **Google Ads is bidding toward a conversion that records zero, and Meta is optimising delivery on an event that fires nowhere.**

The estate is also split in half. GA4 and the Meta Pixel see only the marketing site; PostHog sees only the product. Neither half can answer the question the marketing budget depends on: *which channel brings people who actually study?*

Nothing here is a forecast. Every figure was read from the repository and from live PostHog data on 11 September 2026.

WHAT THIS DOCUMENT ASKS FOR

1. **One security fix, today.** Auth tokens are being written into analytics storage. It is one SDK hook, and it should not wait for a sprint boundary.
2. **One sprint, 79 points, 36 tickets** — of which 13 are P0 and stop money being spent blind. Fifteen of the 36 belong to the `app.nursia.io` team and are a hard dependency.
3. **A decision on the system of record.** The recommendation is PostHog, because it already collects on the side where activation happens and GA4 cannot reach it.
4. **Agreement on six numeric gates** for what "done" means, including 75% attribution coverage and a Meta match quality of 7.5.

What changes if it lands

Budget moves on evidence instead of on click-through rate. A creative cluster can be killed or scaled on whether it produces people who study, not on whether it produces clicks. Smart Bidding and Meta delivery both start learning from a real conversion. And the eight questions in the Outcomes section — none of which can be answered today — become a dashboard somebody checks on a Monday.

What the audit turned up

Every claim below was verified against the repository and against live PostHog data in project 595036 on 11 September 2026. Nothing here is hypothetical.

FINDING 01 • SECURITY • FIX TODAY

Live auth tokens are being stored in analytics.

Supabase returns its session in the URL fragment after OAuth and magic-link signin. The PostHog SDK on `app.nursia.io` captures `$current_url` verbatim, fragment included. In the last 30 days: **49 stored events contain an `access_token` and a `refresh_token`**, and **42 contain a Google `provider_token`**, across 24 people.

A refresh token is a durable credential. Anyone with read access to the PostHog project — staff, contractor, or anyone who obtains a project API key — can lift a session and act as that user, and the Google provider token reaches beyond Nursia entirely. Account emails are captured alongside them in the same strings.

This is not an attribution problem and it does not wait for the sprint. Strip the fragment in the SDK's `sanitize_properties` (or `before_send`) hook on `app.nursia.io`, then purge the affected events and rotate anything exposed. Owned by the app team; raised here because this audit is what surfaced it.

FINDING 02 • ATTRIBUTION • SHIP FIRST

The redirect throws away the query string.

`src/app/(funnel)/signup/page.tsx` is four lines: `redirect("https://app.nursia.io")`. Next issues a 307 to exactly that URL, and nothing on the incoming URL survives. `CtaLink` collects five UTMs plus `gclid` and `fbclid` on the landing page; all of it dies one hop later.

ad click • gclid + fbclid → /lp/meta • CtaLink collects →

/signup?utm_source=...&fbclid=... → ~~307 • params dropped~~ → ~~app.nursia.io • anonymous~~

Google Ads sees no `gclid` at registration. Meta never sets `_fb` on the app domain. GA4 counts the app as a separate session referred by your own marketing site — which is also why organic and paid currently look closer than they are.

FINDING 03 · THE TARGET EVENT · SHIP FIRST

CompleteRegistration does not exist in any system.

You have named it the optimisation target. Today it fires nowhere.

On `nursia.io`, `signedUp()` in `src/lib/analytics.ts` fires the GA4 `sign_up`, the Google Ads *primary* conversion and Meta's `CompleteRegistration` — and its only call site is `SignupForm`, which lives behind the `/signup` redirect stub and never mounts.

On `app.nursia.io`, where registration actually happens, PostHog's event list contains `login_viewed`, `login_completed`, `onboarding_completed` and `qualify_lead` — and **no registration event at all**.

Signups are demonstrably occurring: the captured callback URLs carry `type=signup`. They are simply not recorded as anything.

FINDING 04 · SPLIT BRAIN

Each tool sees exactly half the journey.

PostHog's only `$host` value is `app.nursia.io` — it has never seen the marketing site. GA4, the Meta Pixel and Clarity are installed only in this repo's root layout, so they have never seen the product.

The consequence is that no single system can express the funnel this sprint exists to measure. That is also the opportunity: PostHog is already live on the side where activation and high intent happen, with five days of real data. Adding `posthog-js` to `nursia.io` is the cheapest route to one complete funnel — cheaper than teaching GA4 to cross the boundary.

Moving questions into the app inverts the funnel.

Questions are being withdrawn from `nursia.io`. The measurement loss is small — `FREE_PREVIEW = 2` capped the public preview at two, so the activation threshold of five was never reachable there, and GA4 was recording a teaser rather than practice. Losing it costs little and makes PostHog the single source, which the rest of this document already assumes.

The consequence that matters is order. If questions exist only in the app, they sit *behind registration*:



That deletes the only pre-registration engagement signal there is. The segment it measured — tried it, did not sign up — is exactly the one that separates "the offer is wrong" from "the form is broken", and it is the denominator half of two outcome rows below.

Three knock-on effects. **Meta loses its mid-funnel signal:** the plan sent `question_answered` as `ViewContent` because `CompleteRegistration` alone is too sparse for delivery to learn from early, and app-only questions leave nothing between `PageView` and registration. **The Ads secondary conversion dies:** `NEXT_PUBLIC_ADS_QUESTION_LABEL` fires inside `questionAnswered`, so that action joins the primary at zero. And **the landing pages lose their demo:** `QuestionSet` renders on twelve page types including all six LPs, where "pick an answer — the rationale opens underneath" is the offer. `src/lib/content.ts` states the premise plainly: the SEO pages are the product, and every public page ships real questions in static HTML.

None of that argues the decision is wrong — maintaining two question implementations has a real cost. It argues the decision is a landing-page and SEO change as much as a measurement one, and should be priced as such.

The app entry page is in Google's "poor" band for LCP.

PostHog's `$web_vitals` stream gives a real baseline rather than a lab estimate. `app.nursia.io/` has a **p75 LCP of 4,461 ms** against a 2,500 ms "good" threshold, and a p75 CLS of 0.163 against 0.1. Every paid click lands on a page one hop away from that.

Instrumentation adds scripts, and scripts cost LCP. So the performance budget is written into this PRD as a gate rather than a footnote: no ticket ships if it moves p75 LCP the wrong way.

Who sees what, today

SYSTEM	INSTALLED ON	SEES	BLIND TO	ROLE AFTER THIS SPRINT
PostHog project 595036	app.nursia.io	App funnel, web vitals, replay, autocapture	All marketing pages, all ad traffic	Reporting surface — settled. Full funnel, activation, dashboards, vitals gate
GA4	nursia.io	Content and LPs. Preview questions until they are withdrawn	Registration, activation, purchase	Channel classification and SEO reporting. Possibly cross-domain identity — engineering's call
Meta Pixel	nursia.io	PageView, Lead	The conversion it optimises toward	Delivery optimisation, fed by CAPI
Google Ads	nursia.io	Secondary conversion only	Primary conversion (records zero)	Bidding, on a conversion that fires
Clarity	nursia.io	Heatmaps, session video	Anything past the redirect	Unchanged. PostHog replay covers the app

Two decisions, and only one of them is open

Reporting is settled: the dashboards are built in PostHog. That is a business requirement, not a technical preference — it is where the funnel, the activation rates and the web-vitals gate are specified, and where the team will actually look on a Monday.

Collection is engineering's call, and GA4 is an acceptable answer. If the team can genuinely maintain cross-domain unified analytics in GA4 — the linker, the domain list, the session stitching, and the discipline to keep them working — then take that route. PostHog then serves as the compilation layer: the place every event is assembled into one person and one funnel, and the place the dashboards are built.

One implication is worth stating plainly rather than discovering in week three: a dashboard can only report on data the system holds. **So PostHog has to receive the funnel events whichever route is chosen.** The open question is therefore narrower than it first looks — it is about which system resolves identity across the domain boundary, not about which system the events land in. If GA4 wins the identity question, PostHog still needs the same events alongside it.

Nine areas, each with its own owner and its own gap

The rail on each block is that area's current measurement state: green is instrumented, amber is partial, red is unmeasured.

AREA A

Acquisition & landing

PARTIAL

nursia.io

Six landing pages, each with its own `src` : `/lp/meta` , `/lp/meta-ten-minutes` , `/lp/meta-prioritisation` , `/lp/meta-why` , `/lp/google` , `/lp/reddit` . GA4 and the Pixel both fire here. PostHog does not.

TODAY

GA4 `page_view` , Meta `PageView`

CHANGING

The interactive question is being removed. Every LP needs a new hook, and the page loses its only engagement event

ADD

PostHog `$pageview` , `lp_viewed` with `src`

OWNER

This repo + Ads Manager

GAP

Meta ads are untagged — Paid Social lands in Referral

AREA B

Content & guides

INSTRUMENTED

nursia.io

The best-instrumented area in the estate, and the only one with a proper step-by-step funnel already in place. `resource_offered` fires on visibility rather than render, which is what makes the gate-placement experiment answerable at all.

TODAY

`guide_viewed` , `resource_offered` , `resource_clicked` , `resource_unlocked`

CHANGING

Guides and topic pages lose their embedded questions too — an SEO decision as much as a measurement one

ADD

Mirror to PostHog; send `Lead` via CAPI as well as pixel

OWNER

This repo

GAP

`resource_unlocked` depends on a session, so it breaks past the redirect

AREA C Registration — the target event

UNMEASURED

app.nursia.io

This is the area the sprint exists for. CompleteRegistration is the optimisation target for Meta and the primary conversion for Google Ads, and it fires in neither system. PostHog has no registration event either, despite signups visibly occurring.

TODAY	ADD	PROPERTIES
Nothing. signUp() is unreachable code	registration_completed → PostHog, GA4 sign_up, Meta CompleteRegistration via pixel and CAPI	method, src, utm_*, gclid, fbclid, event_id
OWNER		
App team (contract C-04)		

AREA D Login & returning users

PARTIAL

app.nursia.io

PostHog records login_viewed (101 events, 19 people in 5 days) but only a single login_completed. Either the completion event is misfiring or the login funnel is losing nearly everyone — and today there is no way to tell which, because nothing distinguishes a failed attempt from an abandoned one.

TODAY	ADD	WATCH
login_viewed 101 · login_completed 1	login_failed with a reason; login_method on both	A 1 % viewed→completed rate is almost certainly instrumentation, not behaviour
OWNER		
App team		

AREA E Onboarding PARTIAL

app.nursia.io

`onboarding_step_viewed` fires 26 times across 5 people, but carries no step index in a form the funnel can use, so the drop-off between steps is invisible. `onboarding_completed` has fired twice; `onboarding_skipped` once.

TODAY	ADD	WHY
<code>onboarding_step_viewed</code> , <code>_completed</code> , <code>_skipped</code>	<code>step_index</code> and <code>step_name</code> on every step event	Without an index, a five-step funnel reports as one step repeated
OWNER App team		

AREA F Practice & activation UNMEASURED

app.nursia.io

Activation is five attempted questions, and after the scope change in Finding 05 the app is the only place any of them happen. It is also a place with no question-level event at all: PostHog shows `view_item` and `select_item`, which are not the same thing and carry no correctness. Until NUR-12 lands, practice is entirely unmeasured rather than half-measured.

TODAY	ADD	RULE
GA4 <code>question_answered</code> on nursia.io, capped at 2 — and going away	<code>question_answered</code> in the app; <code>activated</code> once per person at the 5th	Cumulative and cross- surface. <code>question_index</code> cannot substitute — it is per- set and zero-based
OWNER App team (contract C-05)		

AREA G High intent — completed session

UNMEASURED

app.nursia.io

One completed practice session, drill or exam. `drillCompleted()` still fires from `TopicDrill` on the marketing site; `examCompleted()` is behind the `/exam` stub and unreachable. PostHog has `exam_selection_viewed` (1 event) and nothing for completion.

TODAY

Drill only, on the wrong domain. Exam entirely absent

OWNER

App team (contract C-06)

ADD

`practice_session_completed` with `session_type`, `total`, `pct`, `minutes_taken`

META

Custom conversion `HighIntent`, server-side

AREA H Monetisation

PARTIAL

app.nursia.io

`subscription_page_viewed` (8) and `paywall_dismissed` (7) exist, which is enough to see the paywall being rejected and not enough to see anything being bought. There is no purchase event, so ROAS cannot be computed for any channel.

TODAY

`subscription_page_viewed`, `paywall_dismissed`

OWNER

App team (contract C-08)

ADD

`purchase` with `value` and `currency`; Meta `Purchase` server-side

WHY

Without value, every channel looks equally good at the top of the funnel

`$web_vitals` is already flowing into PostHog from the app, which means the budget below can be enforced against real field data from day one rather than against a Lighthouse run. The marketing site has no vitals collection yet — that arrives with `posthog-js`.

TODAY

App vitals live · marketing site uninstrumented

ADD

Vitals on nursia.io; a CI budget check on the LP bundle

CONSTRAINT

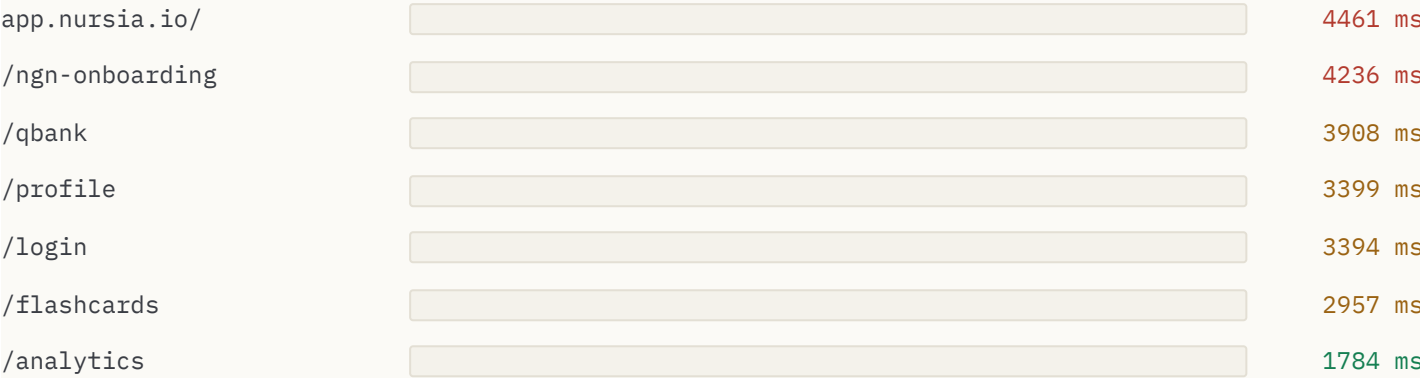
No analytics ticket may regress p75 LCP

OWNER

Both

The performance budget

Measured p75 from PostHog `$web_vitals`, 30 days to 11 September 2026. Bars are scaled against a 6,000 ms ceiling; the “good” threshold is 2,500 ms.



Good ≤ 2500 ms Needs improvement ≤ 4000 ms Poor > 4000 ms

p75 · n = 312 vitals events · 17 people

Surface	Metric	Budget	Current P75	Enforcement
Paid landing pages /lp/*	LCP	≤ 2500 ms	not yet measured	Blocks release of any LP change
Paid landing pages	CLS	≤ 0.10	not yet measured	Blocks release
App entry app.nursia.io/	LCP	≤ 2500 ms	4461 ms	Remediation ticket, not a blocker this sprint
App entry	CLS	≤ 0.10	0.163	Remediation ticket
Every surface	INP	≤ 200 ms	58–66 ms	Currently passing — hold it
Any analytics ticket	ΔLCP p75	≤ +100 ms	baseline above	Hard gate. Ship reverts if breached

How the budget survives four analytics scripts

GA4, the Meta Pixel, Clarity and now PostHog all want to load on the marketing site. Every one of them is `afterInteractive` or later, never `beforeInteractive` — the Clarity tag in the root layout is currently `beforeInteractive` and should move, which is its own ticket. PostHog loads through a same-origin reverse proxy, which also gets it past the blockers that would otherwise bias paid traffic against organic. The LP hero stays server-rendered text and an interactive question, with no video and no client-side image work above the fold.

Google Tag Manager: engineering's call

Raised as a way to protect Core Web Vitals. That specific reasoning does not hold — a container makes load performance worse, not better — but GTM has real benefits that have nothing to do with speed, and there are ways to load it that blunt most of the cost. This section lays out the trade and the options. **The final decision sits with engineering.**

First, the thing that is not true

A container does not replace tags, it loads them. The browser fetches `gtm.js` (tens of kilobytes before a single tag), parses the container, evaluates triggers, and only then injects `gtag.js`, `fbevents.js` and the rest. Those tags still execute on the main thread, just later and behind an extra round trip. Adopting GTM *to improve* Core Web Vitals means adding a script in front of every script already loading.

The real case for it, which is a good one

Autonomy. Marketing adds or changes a pixel without an engineer, a pull request and a deploy. Consent logic, campaign-specific tags and short-lived measurement pixels stop consuming engineering time. If the tag list is about to grow — new ad platforms, affiliate pixels, a consent management platform — that benefit compounds, and it is worth paying some performance for.

THE RISK TO WEIGH, AND IT IS NOT SPEED

A container concentrates failure onto the traffic you are measuring.

`googletagmanager.com` is among the most heavily blocked domains on the web. Today a blocked GA4 request does not take down the Meta Pixel, and neither touches PostHog — failures are independent and partial. Behind one container, one blocked request removes every tag at once.

That failure is not random: it concentrates in privacy-conscious, mobile and paid traffic, which is exactly the segment DOD-1 and DOD-3 count. **This is the item to weigh hardest**, and unlike the speed cost it is largely fixable — see the first-party option below.

Ways to load GTM without paying full price

If the autonomy case wins, these are the approaches worth prototyping. They are not mutually exclusive; the strongest setup combines the first two.

Server-side GTM

A container running on infrastructure you own. The browser makes one first-party request; tags fire server-side and never ship JavaScript to the page. This is the only option that genuinely removes main-thread work rather than rescheduling it, and it solves the blocking problem at the same time.

Best outcome · highest setup cost · needs a hosting decision

First-party container serving

Serve `gtm.js` from your own domain through a Next.js rewrite — the same technique NUR-05 uses for PostHog. Does nothing for main-thread cost, but removes the correlated-blocking risk above, which is the more serious of the two problems.

Cheap · fixes blocking, not speed · do this either way

Deferred container load

Load the container after LCP has settled — on `requestIdleCallback`, first interaction, or a short timer — via `next/script`'s `lazyOnload`. Keeps the container off the critical path entirely. The cost is real: events before the load are missed unless queued, and bounced sessions may never fire at all.

Easy · measurable speed win · costs early events

Web worker (Partytown)

Moves third-party scripts off the main thread into a worker. Next.js exposes this as an experimental `worker` script strategy. Attractive on paper and fragile in practice: tags that touch the DOM or cookies frequently break, and debugging is unpleasant. Prototype before committing to it.

Experimental · verify every tag still fires

A deliberately small container

Most GTM performance complaints are really container-bloat complaints. No custom HTML tags, no unused triggers, no vendor templates kept "just in case", and a periodic prune. A lean container costs a fraction of a typical one.

Free · requires ongoing discipline

Container in front of everything

Migrating GA4, Ads, Clarity, the Meta Pixel *and* PostHog behind one client-side container. This is the configuration that is both slowest and most fragile, and it defeats NUR-05's same-origin proxy. If GTM is adopted, keep PostHog outside it.

Not recommended in any scenario

Recommendation, for engineering to accept or overrule

Not this sprint. With four stable tags and engineers already in the loop, the autonomy benefit is not yet being paid for, and the sprint has a hard LCP gate running through it. Revisit when the tag list approaches ten, or when non-engineers genuinely need to ship pixels — and when you do, evaluate **server-side GTM with first-party serving** rather than a plain client container, since that combination is the one that avoids both problems above.

Whatever is decided, two things hold: `src/app/layout.tsx:138` loads Clarity `beforeInteractive`, ahead of hydration, and fixing that (NUR-21) is worth more than any tag-management change; and PostHog stays on its own same-origin path regardless, because DOD-3 depends on it.

And separately: tags are not why the app entry page is slow

A p75 LCP of 4,461 ms is not recoverable by deferring analytics. Scripts loaded `afterInteractive` compete for bandwidth but do not block the largest contentful paint; removing every tag on that page would not bring it under 2,500 ms. The cause is the hero render, an unoptimised image, or server response time. That is C-12, and treating it as a tagging problem would spend the sprint in the wrong place.

Three things that are requirements, not options

These are not engineering trade-offs to be weighed. They are the conditions under which the rest of the document can be true.

P0 · REQUIREMENT 01

Login lives on nursia.io, not app.nursia.io.

Today `src/app/(funnel)/login/page.tsx` is a redirect stub to the app. That puts the single most identity-rich moment in the journey on the far side of the boundary that destroys identity.

Bringing login back to `nursia.io` is worth more to measurement than any other single change in this document. It is the point at which an anonymous browser becomes a known person, and it is the natural place to call `identify` — on the domain that still holds the first-touch attribution, the `_fbcc` cookie and the click ids. Do it there and the whole cross-domain identity problem shrinks to a handoff of an already-identified user, which is a far easier thing to get right than stitching two anonymous halves together afterwards.

It also fixes Area D on its own: `login_viewed` at 101 against `login_completed` at 1 is currently unexplainable because the completion happens somewhere nothing is watching.

P0 · REQUIREMENT 02

Session recording on the marketing site.

PostHog session replay is already enabled on the product app. It is not on `nursia.io`, which is where every paid click lands and where the drop-off this sprint is trying to explain actually happens.

This is the difference between a number and a reason. The funnel will tell you that Meta traffic drops at the gate; a replay tells you whether they scrolled past it, tapped it and bounced, or never saw it. Clarity covers some of this today, but it cannot be filtered by channel or joined to a person, which makes it useless for exactly the question being asked. Enable it with NUR-05, sampled rather than at 100%, and with the same fragment sanitiser NUR-00 installs.

The token leak is fixed before anything else ships.

Restated here because it is a requirement and not a ticket to be sequenced. 49 stored events hold a live `access_token` and `refresh_token`; 42 hold a Google `provider_token`. Sanitise, purge, rotate. Nothing else in this document should ship first, and no data export or warehouse connection should run until it is done — an export would replicate a credential leak into a second system with its own access model.

Eight calls engineering makes before the sprint starts

Each of these changes what gets built. Left undecided, each has a default that will happen anyway — usually the worse one, chosen by whoever picks up the ticket first. The growth column is why a non-engineer should care about the outcome.

#	CALL	OPTIONS	WHY IT MATTERS COMMERCIALLY	IF NOBODY DECIDES
E-01	Cross-domain unified analytics	GA4 is acceptable if the team can genuinely maintain cross-domain identity there. Otherwise PostHog cross-subdomain identity.	This is what makes one person one person across the redirect. Get it wrong and every activation is credited to "direct", and no channel can be compared to another. Either way PostHog is the compilation layer and the dashboards are built there — so PostHog receives the events regardless of who resolves identity.	Two half-funnels, permanently
E-02	The canonical user id	Supabase user id (natural — it is already the auth system) or something new	The join key for every future question, including anything a warehouse is ever asked. Costs nothing now; costs fuzzy email-matching forever if skipped.	Guesswork joins later
E-03	CAPI transport	Own route handler on Railway, or PostHog's Meta Ads destination, or both	Decides how much is built versus configured. The destination may remove most of NUR-08; the route is still needed for hashed identifiers on registration.	NUR-08 built, possibly needlessly
E-04	Tag manager	None (recommended for now), client GTM, or server-side GTM with first-party serving	Marketing autonomy versus load speed and measurement reliability. See the GTM brief; the recommendation is "not this sprint" and engineering may overrule it.	Status quo, which is fine
E-05	Cookie strategy across the boundary	Apex-scoped .nursia.io cookies, or forward fbclid and mint _fbclid on the far side	Determines whether Meta can attribute a registration at all. The forwarding route is simpler and is what this PRD assumes; apex cookies need both hosts to agree.	Meta attribution stays broken
E-06	PostHog environment gating	Key only (recommended), or NODE_ENV === "production" like the other tags	Whether anyone can verify an event before it reaches production. Not seeing events in development is how instrumentation ships broken.	Copies the other three tags
E-07	Pre-registration engagement signal	Keep one ungated question, or accept the blind spot	Product and growth as much as engineering. Decides whether "tried it, did not sign up" is ever visible again, and whether Meta gets a mid-funnel event. See Finding 05 and NUR-27.	Signal lost silently
E-08	Server-event reliability	Fire-and-forget, retry queue, or a durable queue	A dropped server conversion is a conversion Meta and Google never learn from. Decides whether DOD-3's 75% floor is reachable on a bad day.	Fire-and-forget by default

What is not an engineering call

Three things are settled and should not be reopened at ticket level. **Dashboards are built in PostHog. The Google Ads conversions stay on gtag** — Smart Bidding reads them and nothing else does. And **the six gates in the next section are the definition of done**; if one is judged unreachable, that is a conversation about the gate, not a ticket that quietly ships without meeting it.

DEFINITION OF DONE

Six numeric gates. All six, or the sprint is not done.

Each gate is measurable the day it is claimed, from a named surface. “It looks right in DebugView” is not one of them.

DOD-1 · ATTRIBUTION ≥ 75% Share of <code>registration_completed</code> events carrying a resolvable channel — a UTM pair, a <code>gclid</code>, or an <code>fbclid</code>. Not Direct, not Unassigned. Now: 0% — the event does not fire	DOD-2 · SIGNAL QUALITY ≥ 7.5 Meta Event Match Quality on <code>CompleteRegistration</code>, read in Events Manager. Driven by hashed email, phone, <code>_fbclid</code>, <code>_fbid</code>, IP and user agent on the server event. Now: n/a — no server events
DOD-3 · COVERAGE ≥ 75% Meta-recorded <code>CompleteRegistration</code> as a share of PostHog <code>registration_completed</code> over the same window. PostHog is the denominator and the source of truth. Now: undefined — both sides missing	DOD-4 · DEDUPLICATION ± 2% Pixel and CAPI must collapse to one conversion. Meta's total may not exceed PostHog's by more than 2%, which is the test that catches a broken <code>event_id</code>. Now: n/a
DOD-5 · PERFORMANCE ≤ +100 ms Change in p75 LCP on any instrumented surface, measured on real field data seven days after release. A regression past this reverts the ticket. Baseline: 4461 ms app entry	DOD-6 · HYGIENE 0 Stored events containing <code>access_token</code>, <code>refresh_token</code> or <code>provider_token</code>. Verified by the same query that found them. Now: 49 / 49 / 42

Why 75% and not 95%

Ad blockers, ITP, and people who never consent all remove conversions that no amount of engineering recovers. A realistic ceiling for browser-plus-server capture on consumer traffic is the high eighties; 75% is the line below which the channel comparison stops being trustworthy, because the missing quarter is not missing at random — it skews toward paid, privacy-conscious and mobile traffic. Treat 75% as the floor for believing the dashboard, not as the target to stop at.

The PostHog dashboard

One dashboard, **Growth — channel to activation**, in project `NCLEX-Nursia` (595036). Eleven tiles. Tiles 4, 6 and 10 are the near-term ones; the rest wait on the app contract.

#	TILE	INSIGHT TYPE	DEFINITION	BLOCKED ON
1	Channel funnel	Funnel	<code>\$pageview</code> → <code>registration_completed</code> → <code>question_answered</code> → <code>activated</code> → <code>practice_session_completed</code> , breakdown by <code>utm_source</code> . Note the order: questions now sit behind registration	App contract
2	Activation rate by channel	Trends · formula	<code>activated</code> ÷ <code>registration_completed</code> , weekly, broken down by initial UTM source	App contract
3	High-intent rate by channel	Trends · formula	<code>practice_session_completed</code> ÷ <code>activated</code>	App contract
4	Landing page performance	Trends	<code>\$pageview</code> by <code>src</code> , one line per LP — which creative cluster converts	posthog-js on nursia.io
5	Registration by method	Trends	<code>registration_completed</code> broken down by <code>method</code> (email / google / phone)	App contract
6	Login health	Funnel	<code>login_viewed</code> → <code>login_completed</code> . Currently 101 → 1; this tile is how you find out why	Ready now
7	Onboarding step drop-off	Funnel	<code>onboarding_step_viewed</code> by <code>step_index</code> , then <code>onboarding_completed</code>	<code>step_index</code> property
8	Time to activation	Funnel · time to convert	Median hours from <code>registration_completed</code> to <code>activated</code>	App contract
9	Paywall rejection	Funnel	<code>subscription_page_viewed</code> → <code>purchase</code> , with <code>paywall_dismissed</code> as the leak	purchase event
10	Core Web Vitals by page	Web vitals	p75 LCP / INP / CLS per URL — the DOD-5 enforcement tile	Ready now
11	Activated-user retention	Retention	Cohort on <code>activated</code> , returning on <code>practice_session_completed</code> , weekly	App contract

Two tiles can be built this week

Tile 6 and tile 10 need no new instrumentation — the events are already flowing. Tile 6 in particular is worth building before the sprint starts: a `login_viewed` -to- `login_completed` ratio of 101 to 1 is either a broken event or a broken login, and both are urgent. Build those two first so the dashboard exists and has something real on it while the rest of the sprint fills it in.

Person properties, not event properties

Set the first-touch channel as a *person* property (`initial_utm_source` , `initial_utm_medium` , `initial_fbclid`) on first sight, and never overwrite it. PostHog's person-on-events mode stamps the value as it was at ingestion, so a channel written once at first touch stays readable on every later event — including an activation that happens a week and three sessions after the click that paid for it. Without this, breaking a funnel by channel silently attributes activation to whatever the last session's source happened to be.

Where the server-side events come from

OPTION	COST	CONTROL OF <code>EVENT_ID</code>	OPS BURDEN	VERDICT
Route handler in this repo <code>/api/meta/capi</code> on Railway	None beyond existing hosting	Total — you generate and match it	You own retries	Recommended
PostHog CDP destination Meta Ads Conversions	Included in PostHog usage	Good — PostHog sets it	Lowest of the three	Strong alternative
Meta Conversions API Gateway	~\$40-90/mo cloud	Partial — gateway derives it	Low, opaque when wrong	Reject

Recommendation — own the route, but look at the PostHog destination first

Now that PostHog is the system of record, its native Meta Ads Conversions destination can forward server-side events with no route handler at all, and it already holds the person properties the event needs. That is genuinely less code than NUR-06 and NUR-07 combined. The reason to still own the route is the target event: `CompleteRegistration` has to carry hashed email and phone to reach DOD-2's match quality of 7.5, and that means handling raw identifiers on a server you control. Spike the PostHog destination for `Lead` and the custom conversions; own the route for registration and purchase.

The redirect stub, fixed

Four files, one shape. It keeps its job and stops discarding the reason people arrived.

```
// src/app/(funnel)/signup/page.tsx
import { redirect } from "next/navigation";

const APP = "https://app.nursia.io";

/* Everything the ad platforms and our own funnel need on the far
   side. A bare redirect() drops all of it, which is how a campaign
   ends up bidding toward a conversion it cannot attribute. */
const CARRY = [
  "utm_source", "utm_medium", "utm_campaign", "utm_content", "utm_term",
  "gclid", "fbclid", "src", "guide", "resource", "experiment", "variant",
] as const;

export default async function SignupPage({
  searchParams,
}: {
  searchParams: Promise<Record<string, string | string[] | undefined>>;
}) {
  const from = await searchParams;
  const out = new URLSearchParams();
  for (const key of CARRY) {
    const v = from[key];
    const one = Array.isArray(v) ? v[0] : v;
    if (one) out.set(key, one);
  }
  const qs = out.toString();
  redirect(qs ? `${APP}/signup?${qs}` : `${APP}/signup`);
}
```

And the one-line fix that matters more

On `app.nursia.io`, the PostHog SDK needs a sanitiser before anything else ships. Strip the fragment from `$current_url`, `$pathname` and `$initial_current_url` — Supabase puts the whole session in it, and PostHog is storing it. This is Finding 01, it is one hook, and it should not wait for a sprint boundary.

PostHog on the marketing site

The single change that makes eight of the outcome rows answerable. The product app has been reporting to project 595036 for a while; this side has not, which is why neither half can describe a whole customer. Written out here because the order and the option values both matter, and because two of them are easy to get wrong in ways that cost money rather than throwing an error.

Step 1 — Serve it from our own origin

Two rewrites in `next.config.ts`, plus `skipTrailingSlashRedirect: true`. The asset host and the ingestion host are different and must stay so: the SDK bundle and the replay recorder come from `us-assets.i.posthog.com`, everything else from `us.i.posthog.com`. Pointing both at one destination silently breaks one of them.

WHY THE PROXY IS NOT OPTIONAL

Blocking is not evenly distributed, and it follows the comparison.

`us.i.posthog.com` is on every blocklist worth the name. Paid, mobile and privacy-conscious traffic is blocked at a far higher rate than organic desktop traffic. A dashboard whose entire job is comparing paid against organic therefore cannot use a third-party host: part of the gap it reports would be real and part would be an artefact of who runs a blocker, with no way to separate the two. Same-origin requests are not blocked. This is also what DOD-3's 75% coverage floor depends on.

OPTION	VALUE	WHY THIS VALUE
api_host	/ingest	Same-origin, per the block above. Not the PostHog host.
ui_host	https://us.posthog.com	Where "open in PostHog" links resolve. Without it they point at the proxy path and 404.
capture_pageview	'history_change'	The SDK watches the History API itself, so App Router navigations produce a pageview with no router subscription. The hand-rolled <code>usePathname</code> effect double-fires on first render and misses <code>replaceState</code> .
person_profiles	'identified_only'	Most traffic here never signs up. A profile for every browser that reads one guide costs money and answers nothing.
capture_performance	{ web_vitals: true }	Field Core Web Vitals. DOD-5 is enforced against this, not a Lighthouse run — a lab number cannot tell you what a nurse on hospital wifi sees.
defaults	'2026-08-30'	Pin it. Leaving it unset silently changes behaviour on SDK upgrades.

Step 2 — First touch, written once

Read `utm_source`, `utm_medium`, `utm_campaign`, `utm_content`, `utm_term`, `gclid`, `gbraid`, `wbraid`, `fbclid` and `src` from the landing URL, prefix them `initial_`, and store them with `register_once`.

PostHog sets its own `$initial_*` properties and they are not sufficient alone: a reader who arrives from an ad, leaves, and returns a week later through search has a first touch the browser has already forgotten. More to the point, activation happens days and several sessions after the click that paid for it, and person-on-events stamps values as they were at ingestion — so a channel written once at first touch stays readable on the activation event itself.

THE MISTAKE TO AVOID

Use `register_once`, **not** `setPersonProperties`.

`setPersonProperties` is the obvious-looking call and the wrong one under `identified_only`: it either does nothing, or it creates the person profile for an anonymous reader that the setting exists to avoid. `register_once` stores super properties — written only if absent, so "first touch wins" is true at the storage layer rather than something the code has to remember; attached to every later event from that browser; carried through the eventual `$identify`; and costing no profile in the meantime.

Step 3 — Dual-send the events that already exist

Every analytics call on this site funnels through one `track()` in `src/lib/analytics.ts`. Adding a PostHog capture inside it sends all nine existing events — `guide_viewed`, `resource_offered`, `resource_clicked`, `resource_unlocked`, `question_answered` and the rest — with no change to any of the nine call sites. That single-funnel design is why this step is one edit rather than a sweep.

Step 4 — Environment and verification

`NEXT_PUBLIC_POSTHOG_KEY` holds the project token for `NCLEX-Nursia` (595036). It is public by design, exactly like the GA4 and Meta pixel ids already in `.env.example` — what protects the project is that an ingestion token cannot read data back.

Gate the provider on the key alone, **not** on `NODE_ENV === "production"` as the other three tags are. PostHog is now the system of record, and being unable to see events in development is precisely how instrumentation ships broken — which is the failure this document exists to document.

THE CONSTRAINT ON THIS TICKET

This is the fourth script on the page. DOD-5 applies to it.

PostHog joins GA4, the Ads tag, Clarity and the Meta pixel on every marketing page. Measure p75 LCP for seven days after release and compare against the baseline NUR-22 establishes; a regression past +100 ms reverts the ticket. NUR-21 — moving Clarity off `beforeInteractive`, where it currently executes ahead of hydration — should land first, so the budget has headroom before anything is added to it.

What has to exist before a ticket can close

Every one of these has blocked a sprint somewhere. They are cheap to arrange in advance and expensive to discover on day four with a developer already assigned.

DEPENDENCY	NEEDED FOR	WHO PROVIDES IT	LEAD TIME
Meta system user + CAPI access token	NUR-08, NUR-10, NUR-25	Business Manager admin	Same day, but needs the right admin role
Google Ads API developer token	NUR-24 (offline conversion import)	Google, via the Ads account	Days to weeks. Basic access needs an application – start now
Google Ads OAuth client + refresh token	NUR-24	Cloud console + one consent flow	Hours
A conversion action for purchase	NUR-24	Ads account	Minutes, but must exist before the first upload
PostHog project token	NUR-05 and everything downstream	Already exists – project 595036	None
Supabase service access	E-02 canonical id, later warehousing	App team	Hours
app.nursia.io deploy slot	Twelve tickets in the contract	App team's release process	Unknown – confirm before sprint start
Sanity write token	Content seeding pipeline	Sanity project admin	Minutes
Search Console property + ownership	Sitemap submission	Whoever verified the domain	Minutes if already verified

START THIS ONE TODAY

The Google Ads developer token is the long pole.

Everything else on that list is hours. Basic access to the Google Ads API requires an application that is reviewed by Google, and it is the only dependency here that can take more than a day. NUR-24 – attributing revenue back to the Google click that earned it – cannot start without it. Apply before sprint planning, not after.

Keys the sprint adds

Public values ship to every browser and live in `.env.example` with their real values, exactly as the GA4 and Meta pixel ids already do — what protects them is that they cannot read anything back. Secrets never enter the repository.

KEY	KIND	FOR	NOTES
NEXT_PUBLIC_POSTHOG_KEY	Public	NUR-05	Project token for NCLEX-Nursia (595036). Ships to the browser by design
NEXT_PUBLIC_POSTHOG_HOST	Public	NUR-05	<code>/ingest</code> — the same-origin proxy path, not the PostHog host
META_CAPI_ACCESS_TOKEN	SECRET	NUR-08	System user token. Server only. Never <code>NEXT_PUBLIC_</code>
META_TEST_EVENT_CODE	Dev only	NUR-10, NUR-17	Routes events to Test Events while verifying. Must be unset in production
GOOGLE_ADS_DEVELOPER_TOKEN	SECRET	NUR-24	The long-pole dependency above
GOOGLE_ADS_CLIENT_ID / _SECRET	SECRET	NUR-24	OAuth client for the upload job
GOOGLE_ADS_REFRESH_TOKEN	SECRET	NUR-24	Long-lived. Rotate if it ever reaches a browser or a log
GOOGLE_ADS_CUSTOMER_ID	Internal	NUR-24	Account 6331825613
GOOGLE_ADS_PURCHASE_ACTION_ID	Internal	NUR-24	The conversion action the upload targets

And the ones already in the repository that are not documented

The seeding pipeline reads a dozen variables that appear nowhere in `.env.example` : `CONTENT_DB` , `NURSING_DB` , `NCLEX_DB` , `NCLEX_CSV` , `KW_DIR` , `KW_SEEDS` , `KW_GEO` , `TRENDS_CSV` , `TRENDS_GEO` , `TRENDS_SOURCE` , `TRENDS_ANCHOR` , `TRENDS_HEADLESS` , `TRENDS_PAUSE` , `ADS_CUSTOMER` , `ADS_BROWSER_PROFILE` , `ADS_DOWNLOAD_DIR` and `SIGNIN_WAIT_MINUTES` . Nobody but the person who wrote the pipeline can run it today. Documenting them is NUR-31 and costs an hour.

Sanity Studio

The Studio under `studio/` is the CMS behind the guides, the nursing library and the topic pages, and it now ships in the repository with its schema. Three things it needs that are not yet settled: a **deployed Studio URL** so editors do not run it locally, the **write token** scoped to the seeding pipeline rather than shared, and the **revalidate webhook** pointed at `/api/revalidate` with `SANITY_REVALIDATE_SECRET` set on both sides — without which an edit reaches the live site on the next deploy rather than in seconds.

Purchase, and how it reaches each platform

Registration is the optimisation target. Purchase is the event that says whether the target was worth optimising toward — and it happens on `app.nursia.io`, several sessions after the click, where no browser tag from this repo will ever run.

PLATFORM	EVENT	TRANSPORT	KEYED ON	WHY THIS TRANSPORT
PostHog	purchase	SDK, client	person_id	System of record. Carries <code>value</code> , <code>currency</code> and plan, against a person who already holds the first-touch channel.
Meta	Purchase	CAPI, server	_fbclid / _fbclid / hashed_email	The browser is not trustworthy for a payment event and iOS strips most of it. Server-side is the only path that reaches DOD-2.
Google Ads	Purchase	Enhanced Conversions, or Zapier	gclid / gbraid / wbraid	The conversion happens where the Ads tag never loads, hours or days after the click. Neither route needs API access — see the correction below.
GA4	purchase	Measurement Protocol	client_id	Optional, and only worth it once cross-domain measurement is configured. Otherwise the session is already broken.

The Google Ads API is not on the critical path.

An earlier version of this document made the Google Ads API developer token a P0 dependency and called it the long pole, on the grounds that its review takes days to weeks. That was wrong, and it would have put a multi-week approval in front of a problem a spreadsheet solves in an afternoon.

Offline conversion import does not require the API. Google Ads accepts the same `gclid` + timestamp + value payload from a manual upload, a scheduled Google Sheet, or a Zapier action — none of which need a developer token or an approval. There are four routes and only one of them is gated:

ROUTE	DEV TOKEN	WHEN IT FITS
gtag + Enhanced Conversions on the purchase page	Not needed	Simplest, once <code>gclid</code> survives NUR-01. Start here
Zapier offline conversion	Not needed	Webhook from purchase → Zapier → Ads. Also covers Meta offline conversions with the same pattern
Scheduled Google Sheets import	Not needed	Fine at current volume. Least code of the three
Ads API offline conversion import	Required	Only when upload frequency makes the others a chore, or conversion adjustments and retries need handling programmatically

So the developer token moves from P0 to P2: apply when volume justifies automation, not before. At 24 users and five days of data, building an automated upload pipeline is building for a volume that does not exist yet.

Whichever route: it is an upload, not a container

This does not reopen the GTM decision. That rejection was about a *client-side* container loading tags in a browser. What purchase needs is the opposite of a browser tag: an upload that never touches one.

Persist `gclid` at first touch — NUR-01 forwards it, C-02 stores it — then upload the conversion through the Google Ads API with the click id, the conversion action, the timestamp and the value. It is immune to ad blockers, immune to ITP, and works however many days later the purchase lands. **Enhanced Conversions for Web** is the fallback for purchases where no click id survived, matching on hashed email instead.

One thing the current `CtaLink` misses: iOS app-campaign traffic arrives carrying `gbraid` or `wbraid` rather than `gclid`. Both have to be carried and stored, or every iOS purchase is unattributable no matter how good the upload is.

APPLIES TO EVERY CONVERSION, NOT ONLY PURCHASE

A conversion without a value can only be counted, never weighed.

Both Meta and Google can bid toward *value* rather than volume, and both ignore the option entirely if the events arrive without a `value` and a `currency`. Count-based bidding treats every registration as identical, which is precisely the assumption this whole document exists to disprove.

`purchase` carries the real amount. Everything upstream carries a **modelled** value — the expected revenue of reaching that step — so delivery can prefer the registration that becomes a customer over the one that does not. This matters most right now, because purchase volume is far too low to optimise against directly; a modelled value on registration is what lets the platforms learn months before purchases alone would.

EVENT	VALUE	DERIVATION
registration_completed	Modelled	Purchase value × observed registration→purchase rate
activated	Modelled, higher	Same, conditioned on having activated. This is the step that predicts revenue
practice_session_completed	Modelled, higher still	Same, conditioned on high intent
purchase	Actual	The amount charged. Never modelled

Two cautions. **A wrong model is worse than a flat one:** until there are enough purchases to measure the rates honestly, send one conservative flat value per step rather than a confident guess, and revisit once DOD-1 passes. And **set `currency` explicitly on every event** — a missing currency is treated as the account default, which silently mixes denominations the first time pricing changes.

One `event_id`, four destinations

The id generated for the PostHog event is also the Meta `event_id` and the Ads `order_id`. It is what stops a purchase counting twice when both the browser and the server report it, and what lets the four systems be reconciled against each other afterwards — which is the whole of DOD-3 and DOD-4.

Use GA4's own names, or its reports stay empty

GA4 recognises a specific set of event names and builds reports on them automatically. An event called `registration_completed` is a custom event: it works in Explorations and populates nothing else. The same event called `sign_up` lights up User Acquisition, Conversions and the default funnel without configuration. So the wire name is GA4's; the internal name can be whatever the code prefers.

MOMENT	GA4 NAME	REQUIRED PARAMETERS	WHAT IT UNLOCKS IN GA4
Account created	sign_up	method, value, currency	Recognised conversion; User Acquisition reporting by channel
Returning session	login	method	Returning-user segmentation
Onboarding begins	tutorial_begin	–	Pairs with tutorial_complete for a built-in completion rate
Onboarding finishes	tutorial_complete	–	As above
Paywall viewed	begin_checkout	value, currency, items	Enters the standard ecommerce funnel
Payment taken	purchase	transaction_id, value, currency, items	Monetisation reports, ROAS, LTV. None of it works without transaction_id and value
Refund	refund	transaction_id, value, currency	Keeps revenue honest rather than cumulative

`transaction_id` is what stops revenue being counted twice

GA4 deduplicates purchases on `transaction_id` the way Meta deduplicates on `event_id`. Without it, a customer who refreshes a confirmation page books the revenue again, and every downstream number — ROAS, LTV, value-based bidding — inherits the error. Use the same id across all four systems: PostHog, GA4 `transaction_id`, Meta `event_id`, Ads `order_id`.

Onboarding, measured step by step

`onboarding_step_viewed` has fired 26 times across 5 people and carries **no step index**. A five-step funnel therefore reports as one step repeated five times: you can see that people are in onboarding and never where they stop.

EVENT	PARAMETERS	FIRES	STATUS
onboarding_started	source, step_total	First step rendered, once per person	Missing

EVENT	PARAMETERS	FIRES	STATUS
onboarding_step_viewed	step_index, step_name, step_total	Each step shown	Exists, no index
onboarding_step_completed	step_index, step_name, seconds_on_step	Each step advanced past	Missing
onboarding_completed	step_total, minutes_taken	Final step done	Exists — 2 events
onboarding_skipped	last_step_index, last_step_name	Abandoned or dismissed	Exists, no index

Viewed and completed, not just viewed

The pair is the point. `step_viewed` alone cannot distinguish someone who read a step and moved on from someone who read it and quit — both produce exactly one view event. With the completion event, the drop at each step is the difference between the two, and `seconds_on_step` separates a confusing step from a boring one. That is what turns Area E from "26 events" into an answer about which step to rewrite.

Where it lands

All of this is `app.nursia.io` and therefore contract work — C-09 on the app contract sheet. In GA4 it rides the `tutorial_begin` / `tutorial_complete` pair above; in PostHog it becomes dashboard tile 7, a funnel broken by `step_index`. Neither is buildable until the index exists, which is why a one-point ticket blocks a whole area.

What you should be able to see, and what you would do about it

The ticket list lives in the tracker, where it can be reordered without reissuing this document. What belongs here is the reason for it: eight questions the business cannot currently answer, the data that answers each, and the decision it unblocks. A ticket that serves no row below does not belong in the sprint.

QUESTION	WHAT YOU LOOK AT	WHERE	DECISION IT DRIVES	TODAY
Which channel brings people who actually study?	Activation rate — <code>activated</code> over <code>registration_completed</code> — split by first-touch channel	Tile 2	Move budget between Meta, Google and organic. High signup with low activation means a channel is buying clicks, not users.	Cannot answer
Which ad creative earns its spend?	Registrations and activation by <code>src</code> , one line per creative cluster	Tile 4	Kill or scale a cluster on downstream quality instead of on click-through rate.	Cannot answer
Is Meta's delivery learning from the right signal?	<code>CompleteRegistration</code> volume and event match quality	Events Manager	Whether to hold the target at registration or move it deeper, to <code>activated</code> , once volume supports it.	Cannot answer
What is a registration worth, by channel?	Purchase value over registrations, by first-touch channel	Tile 9 + revenue	Set a target cost per registration per channel, rather than one blended number that flatters the worst one.	Cannot answer
Where do paid users drop out of the product?	The five-step funnel, filtered to paid traffic only	Tile 1	Fix the step, not the ad. A drop at onboarding is a product ticket; a drop at registration is a landing-page one.	Cannot answer
Is login broken, or is the login event broken?	<code>login_viewed</code> to <code>login_completed</code> , with <code>login_failed</code> carrying a reason	Tile 6	Ship a login fix or an instrumentation fix. 101 views produced 1 completion and nothing distinguishes the two cases.	Half — no failure reason
Are we buying clicks that land on a slow page?	p75 LCP by landing page, cut by channel	Tile 10	Choose between performance work and creative work. A 4.4 second entry page is not a copy problem.	App side only
Which content pulls people into practice?	<code>guide_viewed</code> through to <code>activated</code> , by guide and topic	Tiles 1 + 4	Decide where the next twenty guides go. The whole content programme rests on this row.	Half — breaks at the redirect
Did someone try the product and decline to sign up?	An engagement event that fires before registration, against <code>registration_completed</code>	Tile 1	Separates "the offer is wrong" from "the form is broken" — two problems with nothing in common except the number they move.	Going away — see Finding 05

The order these arrive in

Rows six and seven are answerable this week on data already flowing — two dashboard tiles, no new instrumentation. Rows one to five all wait on the same two things: attribution surviving the redirect, and `registration_completed` existing at all. Everything in the P0 block serves those two, which is what makes it the P0 block.

Delivery

Thirty-six tickets across six workstreams, sequenced by dependency, live on the *Sprint board* sheet of `nursia-analytics-sprint-tracker.xlsx` with owners, points and status. P0 is now NUR-00 to NUR-10 plus NUR-28 to NUR-30: the security fix, the redirect, the target event, the Ads conversion, ad tagging, PostHog on the marketing site, and the whole Meta Conversions API path including hashing and deduplication. CAPI sits in P0 rather than P1 because `CompleteRegistration` sent from the browser alone cannot reach a match quality of 7.5, and DOD-2 is not optional.

What could still make these numbers lie

RISK	EFFECT	MITIGATION
The app team does not implement the contract	Areas C, F, G and H stay unmeasured regardless of anything in this repo	Fifteen of thirty-six tickets are theirs. Confirm ownership before sprint start.
Dedup misconfigured	Every conversion counts twice; bidding over-corrects	DOD-4 gates the sprint at ±2%.
Meta ads stay untagged	Paid Social reports as Referral; the channel split is silently wrong	NUR-04 is one point and blocks the whole dashboard.
Four analytics scripts on one page	LCP regresses on exactly the pages bought with ad money	DOD-5 is a hard gate measured on field data, and NUR-21 buys headroom first.
Consent and PII	Hashed email sent without a lawful basis — on top of the tokens already stored	NUR-00 first. Hash server-side only, document the basis before NUR-09 ships.
Removing try-before-signup	Signup rate falls. The LP copy currently promises free questions, and the demo is the offer	Price it as a landing-page change, not a cleanup. Measure signup rate against the pre-change baseline before scaling spend.
No pre-registration engagement event	"Interested but did not sign up" becomes invisible, so an offer problem and a form problem look identical	Consider keeping one ungated question purely as an engagement signal, or accept the blind spot knowingly.
Five days of data	Every baseline here rests on 23 people in a pre-launch window	Re-measure all baselines once real traffic arrives; treat current figures as directional.

Companion workbook: `internal/doc/nursia-analytics-sprint-tracker.xlsx` — sprint board, definition-of-done gates, event QA matrix, PostHog dashboard build sheet, performance budget and the app.nursia.io contract checklist. Repository claims verified at commit `d37a534` ; PostHog figures queried live from project 595036 on 11 September 2026, covering 7–11 September, 6,103 events across 24 people.